

Capture, Verify, Govern: Decision Provenance for Agent Artifact Handoffs

Anonymous Authors

Abstract—A tool-call log, a matching file hash, and a reuse authorization answer different questions. When one agent continues another’s work it inherits decisions as well as files; a recorded decision need not be supported by its evidence, and a supported decision need not be authorized for reuse. We present a capture, verify, govern framework that separates these responsibilities and specifies the information needed at each boundary. TRACE, an open decision-provenance record, holds submitted decisions; a domain-specific verifier checks their evidential consistency; and Proofpress, an evidence ledger, keeps intake apart from claim admission and governed reuse. We instantiate the verification layer on RSI-Exam program-version selection: in twelve synthetic packages, format checks reject one of nine authored faults, file-binding checks reject three, and full checks reject nine; two valid controls and a consistent unsigned reward rewrite pass. A separate local demonstration imports three TRACE decisions as evidence without creating claims or admissions; because the intake projection does not carry the estimate, a verifier-rejected numerical error imports as evidence too, so domain verification must remain a separate step. These results characterize verification and intake boundaries, not the effectiveness of a complete governance workflow. Recorded is not verified, consistent is not authentic, and neither is authorized for reuse.

Index Terms—decision provenance, agent handoffs, verification, governed reuse

I. INTRODUCTION

A tool-call log, a matching file hash, and a reuse authorization answer different questions. Suppose one agent hands a revised program to another agent or researcher in a different trust domain. The recipient can inspect the files but cannot observe the producing runtime, and needs to know why this version was selected and whether it is appropriate to rely on that decision now. Treating any one of the three signals as blanket trust obscures what the recipient can actually check.

Our running example is an AI coding agent improving a program that plays 2048: it edits, tests, and keeps a version or returns to an earlier one. RSI-Exam provides this setting: agents improve task programs using visible evaluations, then submit them for grading on unseen data [1]. Here, self-improving means improving the task program, not the model’s weights. A kept version can become the parent of later edits. Its successor inherits the consequences of that choice without necessarily retaining the producing runtime.

We ask: *how should a submitted decision become checkable evidence, and what must remain separate before downstream reuse?* The audit unit is an explicit decision and its supporting artifacts, not every tool call or inferred private rationale. We propose three responsibilities: *capture* the submitted decision; *verify* its domain-specific evidence relationships; and *govern*

whether a bounded claim may enter a recipient’s context. No stage automatically confers the authority of the next.

Our contributions are (1) this separation and an evidence contract for crossing its boundaries; (2) a partial instantiation using TRACE, an RSI decision verifier, and Proofpress, with the implemented and unimplemented parts of each boundary stated (Table I); and (3) a controlled characterization of the verifier plus a local demonstration of evidence-only intake. The framework is a design contribution, not a claim that its full workflow has been empirically validated. We test packages and local interfaces, not a multi-agent network, human-review outcomes, or improvements in downstream performance.

A. A real record shows the evidence gap

One rollout in our own RSI-Exam instrument campaign makes the problem concrete. Its record, checked against the full retained job directory, passes every check with complete coverage: four recorded gate decisions, two candidates each screened and then confirmed on fresh seeds, every interval recomputed from the bound per-seed results. The record still cannot say why candidate v2 was reverted: it reports a higher visible mean than its parent, 48115 versus 38974.5, but the agent reverted it without consulting the gate, so no evaluation under the selection rule was recorded. The higher score alone cannot explain the choice, nor prove that reverting was wrong. A public export of the same rollout retains only the capsule and the log, so its recipient can verify the log’s hash and recover the branch history but cannot recheck an interval. What is handed over decides what can be checked.

In another rollout, under the exam’s own program, the agent kept nine of nine candidates; scored afterwards on sealed seeds, four of those keeps made the program worse, and the run’s log records each keep but not why. These real-run examples motivate what must be captured; the controlled experiment below tests the verifier on a different, fully supplied synthetic package.

II. THE CAPTURE, VERIFY, GOVERN FRAMEWORK

A. Responsibilities and the evidence contract

The producer may omit decisions, supply mismatched artifacts, or coherently fabricate an entire package. The receiver has the disclosed bytes, a chosen checker, and its own reuse policy; it has no trusted witness to execution. The framework makes these limits visible rather than reading a successful import or check as general trust.

At the governance boundary (Table I) the admitting principal may be a human, a service, a quorum, or a policy

TABLE I

THE EVIDENCE CONTRACT PER BOUNDARY: REQUIRED OF THE TRANSFERRED OBJECT, IMPLEMENTED HERE, AND NOT IMPLEMENTED. RECORDED IS NOT VERIFIED, CONSISTENT IS NOT AUTHENTIC, AND NEITHER IS AUTHORIZED FOR REUSE.

Stage	Required of the transferred object	Implemented here, and not
Capture	Stable decision identity; distinct proposer and resolver, so a proposal is never resolved by its proposer; disposition; revision links; digest-bound artifact references; the claimed rule. Unsubmitted choices stay outside.	TRACE record with an RSI task profile. Not established: correctness, complete capture, authenticated authority (a recorded resolver is attribution).
Verify	A check result naming the checked package, checker and profile revision, outcomes, diagnostics, and coverage scope; a bare “pass” cannot say which decision was checked.	Offline verifier: recomputed paired statistics, decision rules, submitted-version identity, coverage. Not established: that an execution produced the measurements.
Govern	A bounded claim, its reuse scope, references to evidence and check results; a check result bound to the exact decision and package so that policy can require it; an admitting principal independent of the proposer.	Proofpress: evidence-only intake, separate claim proposal. Not implemented: binding the check result to the candidate for policy enforcement; policy-enforced admission.

mechanism; this instantiation’s policy assigns it to a human, and under that policy the producing agent must not admit its own claim. A downstream consumer should retrieve only admitted, in-scope context; admission is not permanent proof of truth.

This contract is a composition rule, not a new universal wire schema. Capture, checking, and admission may be implemented by different systems. Their statuses must not be silently promoted: TRACE accepted, RSI keep, and Proofpress admission have different meanings.

B. Partial implementation and its present boundary

TRACE [7] is an open-source (Apache-2.0) decision-provenance record for human and AI work; several unrelated systems share the name. Each decision carries a proposer, a resolver, a disposition, a rationale, revision and correction links, and evidence digests; an AI may not resolve its own proposal. Proofpress [8] is an open-source (Apache-2.0) evidence ledger in which imported evidence, proposed claims, and admitted claims are distinct objects. The RSI decision gate records a proposing agent and a *system* resolver. A converter maps keep/revert/pending and confirmation links into a TRACE record (schema version 0.5.1, the version the intake adapter pins). It checks record-level constraints but does not recompute the measurements. Separately, the offline verifier consumes the full capsule and files. TRACE is a representation of submitted decisions, not a replacement for that package.

Proofpress imports a bounded projection of TRACE as external evidence. Within a confidence block it retains interval, method name/resample count, sample size and named evidence digests; it drops other structured rule fields such as estimate, threshold and bootstrap seed. It neither opens the referenced

result files nor consumes a verifier success as an admission instruction. Claim proposal, review and context retrieval are separate operations.

This is not yet one automatically enforced pipeline from verified package to approved context: the TRACE adapter does not require a receipt proving that the exact imported material passed the RSI checker, and a deployment whose admission policy requires that receipt must bind and check it. The demonstration below tests this intake boundary and stops before human approval.

III. INSTANTIATION: RSI DECISION VERIFICATION

A. What the producer hands over

The evidence package contains saved code versions, test-result files, a decision log, submitted code, and a machine-readable summary called a *capsule*. The producer records decisions during the run and builds the capsule afterward. The recipient checks this directory without accessing the agent’s runtime or rerunning its program.

Each decision identifies five things: the *parent* version being compared against and the new *candidate*; the parent and candidate result files, labeled by role and bound by file hashes; the score direction (higher or lower is better), statistic, confidence-interval procedure, random seed, and threshold; the recorded calculation and decision (keep, revert, or await a confirmation evaluation); and any later confirmation decision that resolves a pending state. The capsule also links the submitted program to a recorded version. A version occurrence is distinct from its code contents: reverting can create another occurrence of the same code. Code identity follows the task’s staging rules, excluding Python bytecode caches; a separate whole-directory hash checks the supplied tree. These conventions avoid confusing a cache change with a program change.

B. A worked decision, from scores to submission

Table II follows the synthetic package. v1 is the parent. Candidate v2 performs worse and is reverted. Candidate v3 is also derived from v1: its initial result is inconclusive, but a subsequent confirmation supports keeping it. The submitted program is v3.

TABLE II

RUNNING EXAMPLE FROM THE SYNTHETIC PACKAGE OF SECTION IV. EACH ROW COMPARES THE CANDIDATE WITH v1 OVER EIGHT PAIRED GAMES; 90 PERCENT PERCENTILE BOOTSTRAP, 5000 RESAMPLES, SEED 20260902.

Candidate/test	Mean delta	90% interval	Decision
v2 / initial	−318.75	[−382.5, −258.75]	Revert
v3 / initial	260	[−30, 583.75]	Pending
v3 / confirm	518.75	[467.5, 575]	Keep

The calculations use paired games: parent and candidate are evaluated on the same seed within each comparison. For a higher-is-better score the mean improvement over the n paired seeds is $\bar{\Delta} = \frac{1}{n} \sum_{i=1}^n (s_{\text{cand}}(i) - s_{\text{par}}(i))$, and the interval expresses uncertainty around it. The verifier recomputes the estimate and percentile-bootstrap interval from the supplied per-seed results

and declared algorithm parameters. For a lower-is-better score the differences change sign.

In this package’s declared policy, an interval entirely below zero leads to revert, one entirely above zero supports keep, and an interval spanning zero requires confirmation. Thus the initial v3 result cannot by itself close the decision. Removing its confirmation while still submitting v3 leaves an unresolved pending decision. More generally, the verifier checks the recorded policy, not whether that policy is scientifically optimal.

C. What the recipient checks

The checks answer progressively stronger questions. **Format:** are required fields present and well formed? **File binding:** do referenced files exist, with bytes matching their declared hashes? **Decision consistency:** do parent and candidate references, recalculated statistics, decision rules, and submitted-version identity agree? Full verification also reports *coverage*: whether every snapshot in the supplied version directory appears in the record. An unrecorded extra snapshot can leave the other checks passing but make coverage partial. Our experiment requires complete coverage for acceptance. This is completeness of the supplied directory, not proof that the producer disclosed every experiment.

Consider changing v2’s recorded mean delta from -318.75 to -317.75 , in both the decision log and capsule, then updating the log hash. Format and file binding still pass. Recalculation detects the discrepancy. This particular one-point error would not change the revert verdict; it tests numerical consistency, not the discovery of an incorrectly chosen winner. A separate test changes the submitted code while retaining the old version claim, exercising decision-to-artifact identity.

IV. EVALUATION AND INTEGRATION DEMONSTRATION

A. What is being tested?

We test the checking procedure, not the agent’s ability to play 2048. No new agent rollout is used for this experiment. We copy the committed synthetic package twelve times, apply one specified change per case (none for the clean control), and require each copy to pass full verification before modification.

The twelve cases comprise nine deliberately introduced faults, two valid controls, and one authenticity-boundary control. Their manifest was committed before the results were committed, but after inspecting the implementation and development tests. This is an author-designed characterization, not an independent or held-out benchmark. The implementation is unchanged by the study.

We run three checking configurations on each package: **S** checks format only; **B** adds file bindings; **V** performs full verification and requires complete coverage. B uses the production hash functions but omits recalculation, decision rules, cross-field identity and coverage. These are internal ablations of our checker, not measurements of competing systems.

To avoid testing only stale hashes, relevant decision changes are mirrored in the log and capsule and the log hash is refreshed. Swapping the parent/candidate measurements exchanges the referenced paths and hashes while retaining their role labels.

TABLE III
CONTROLLED PACKAGES. S: FORMAT; B: FORMAT + FILE HASHES; V: FULL CHECKS INCLUDING COVERAGE. A: ACCEPTS THAT CHECK SCOPE; R: REJECTS.

Package change	S	B	V
None (valid control)	A	A	A
Excluded caches differ; full hashes refreshed	A	A	A
Reward and declaration rewritten together	A	A	A
Unrecognized version status	R	R	R
Referenced measurement file removed	A	R	R
Recorded score direction reversed	A	A	R
Recorded mean delta increased by one	A	A	R
Parent/candidate measurements exchanged	A	A	R
Pending submission left unconfirmed	A	A	R
Submission changed; version claim retained	A	A	R
Recorded snapshot removed	A	R	R
Unrecorded snapshot added	A	A	R

Changing the submitted program also refreshes its hashes while retaining the claimed version. All cases, changed-file hashes and actual diagnostics are recorded in the study’s machine-readable results.

B. Results and interpretation

Table III reports every outcome. S rejects one of nine faults, B rejects three, and V rejects nine. The six additional refusals cover three measurement inconsistencies, an unresolved confirmation, a submitted-version mismatch, and an incomplete snapshot inventory. Thus files can match their declarations while the relationships needed to audit a decision still fail.

Both valid controls pass: the unchanged package and one whose excluded bytecode caches differ. This establishes acceptance on two examples, not a population false-positive rate; likewise the fault counts do not estimate detection rates on naturally occurring errors.

Rejection reasons matter. The extra snapshot is rejected for partial coverage, not an integrity error. The missing snapshot is correctly refused, but its diagnostic prefix differs from the manifest’s expected prefix. All nine faults are refused, but only eight match the prespecified diagnostic target. The unchanged expectation and actual output are both preserved; unexpected runtime exceptions abort the study rather than count as detections.

C. A passing rewrite defines the trust boundary

The final control changes an unsigned final reward file and the capsule’s matching reward to 0.5, then refreshes the file hash. V accepts. This is the final benchmark reward, not one of the paired visible-game measurements used for keep-or-revert decisions. The verifier checks agreement with the supplied reward file; it neither authenticates the grader nor recalculates that reward from underlying score details.

This accepted rewrite bounds the interpretation of the nine refusals. Our mechanism checks consistency among supplied declarations and bytes. A producer able to rewrite evidence consistently can still supply false material. Acceptance must therefore not be read as proof that these experiments really happened or that the agent is trustworthy.

D. Evidence intake does not grant reuse

In a separate local fixture demonstration, we check the clean RSI package, convert its three decisions to TRACE records, and import them into a disposable Proofpress repository. The committed runner pins a public Proofpress revision and contacts no hosted service or agent. The import creates three source records and three evidence items, but zero claims, zero admissions and an empty governed-context result. Re-import leaves the ledger unchanged. An unsupported version, malformed interval and changed normalized content under an existing identity are refused.

We then reuse the one-point mean-delta fault from the controlled study. The full verifier rejects it, but the converter accepts it and a fresh Proofpress receiver imports it as evidence. This is expected: intake validates its bounded projection, not the measurement calculation. The estimate lies outside the adapter's projection, so intake cannot inspect this discrepancy even in principle; the case places responsibility for domain verification, it does not measure detection. Finally, explicitly proposing a synthetic claim creates one candidate but still zero admissions and no governed context. No human approval or downstream agent run is exercised.

In this implementation these observations demonstrate two non-equivalences: importable evidence is not necessarily verified evidence, and even an explicit candidate is not authorized context. They support interface separation, not the effectiveness of human governance; this run tests conversion and intake, not upstream schema conformance.

V. RELATED WORK AND LIMITS

PROV-CONSTRAINTS defines consistency conditions over provenance histories [2]; PROV-AGENT extends W3C PROV to agent interactions in scientific workflows [3], and TRACE exports a proposed PROV mapping whose round-trip fidelity is not established. We do not claim that provenance or the separation of concerns is new; recent work states the separation independently, distinguishing what a hash, a signature, and an anchor can and cannot establish [4]. Our contribution is the composition around one handoff, a decision-level record, an executable task-specific check that recomputes the selection rule from bound measurements, and a non-promoting intake, with the counterexample to silently promoting recorded into consistent into admitted. The RSI profile supplies paired-result recalculation, confirmation state and decision-to-submission identity; another domain needs its own checker.

NovaFabric seals and replays whole agent runs as tamper-evident capsules [5]; a replay does not check a selection rule, which our verifier recomputes. Audits of self-improving agents find that tampering with the agent's own harness occurs in real runs and often persists in the lineage of the best agent [6]; where they audit harness edits, we recompute whether each kept version was supported. Supply-chain frameworks such as in-toto verify cryptographically linked steps [9]; our unsigned packages have no comparable authentication. None of these is a baseline here. Storage, authentication, and decision-consistency checks address different parts of a handoff and compose: neither a

signature nor a consistency check alone establishes that a reward is correct.

Our results use one synthetic base, author-selected changes and shared checking components. The two real rollouts motivate capture and are not experimental results; the public export of the first cannot be fully replayed. The integration demonstration is local and stops at an unadmitted candidate. We evaluate one handoff boundary, not decentralized identity, cross-domain authorization, or coordination among agents. We have not measured capture completeness, human-review quality, audit effort, downstream agent benefit, cross-domain generalization or resistance to coordinated falsification. Reproducing an interval does not certify its validity under adaptive reuse of seeds; a rule-consistent decision can perform poorly on unseen games; running checks outside the producing runtime does not authenticate the files that runtime supplies. Binding a checker's result to a candidate so that an admission policy can require it is the next step here, with network identity and a complete multi-agent governance evaluation.

VI. CONCLUSION

Capture, verify, govern separates what was submitted, what can be checked, and what a recipient may rely on. Of the nine authored faults, full verification refuses the six that format plus file binding accept, while accepting a consistent unsigned rewrite. The TRACE-to-Proofpress intake imports evidence without automatic claim admission. These bounded results support the separation of responsibilities, not validation of the entire framework. Recorded is not verified, consistent is not authentic, and neither is authorized for reuse.

ACKNOWLEDGMENT

An AI coding assistant was used to suggest framing and draft text in Sections I, II, and V and to generate parts of the code for the study and demonstration in Section IV. The authors specified the studies, revised the text, reviewed the code, reran the reported checks, and take responsibility for the manuscript.

REFERENCES

- [1] Aiming Lab, *RSI-Exam*, 2026. [Online]. Available: <https://github.com/aiming-lab/RSI-Exam>
- [2] J. Cheney, P. Missier, and L. Moreau, Eds., *Constraints of the PROV Data Model*, W3C Recommendation, Apr. 2013.
- [3] R. P. Souza *et al.*, "PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows," in *Proc. IEEE Int. Conf. e-Science*, 2025, doi: 10.1109/escience65000.2025.00093.
- [4] A. Brömme, "An evidence model for agentic processes: Evidence claims, trust assumptions, and policy assessment," arXiv:2609.08481, Sep. 2026.
- [5] M. S. Ardebili, "NovaFabric: Tamper-evident, replayable evidence for autonomous AI agent runs," arXiv:2609.12582, Sep. 2026.
- [6] X. Wang, X. Zhang, and J. Shao, "Auditing harness tampering in self-improving agents," arXiv:2609.00069, Aug. 2026.
- [7] *TRACE: A decision-provenance record for human and AI work*, v0.5.1, Apache-2.0, 2026. Repository and DOI withheld for review.
- [8] *Proofpress: An evidence ledger with separate claim review and governed reuse*, v0.4.0, Apache-2.0, 2026. Repository withheld for review.
- [9] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, *in-toto: Providing farm-to-table guarantees for bits and bytes*, in *USENIX Security*, 2019, pp. 1393–1410.